In our case, a “visible” value, where visible is true, means that the target is not hidden.

To use values, we also have to declare them in our controller. Again, sigh, twice. The declaration is another static property, values, but rather than an array like the other ones we’ve seen, the values property contains an object with the type of each individual value we want to use; at the moment ours looks like this:

chapter_03/05/app/javascript/controllers/favorite_toggle_controller.ts

```
static values = { visible: Boolean }
visibleValue: boolean
```

The key in this object visible means we’ve declared a visible value in the markup, and the Boolean means we expect the visible value to be a Boolean, so Stimulus will automatically convert the string value to a Boolean one.

The types that Stimulus makes available are Array, Boolean, Number, Object, and String. Boolean and Number are converted from the string using the toString method, while Array and Object are converted using JSON.stringify. If a value isn’t specified, the default is an empty object, array, or string; the number 0; or Boolean false.

Declaring a value gives you three properties, similar to what we’ve seen for targets and classes. For the singular (non-array) properties, there’s a property, <valueName>Value, that can be used as a getter and also as a setter, as in visibleValue = true. There’s also an has<valueName>Value, which returns true or false depending on whether the attribute is actually present in the DOM. The array properties are the same but plural: <valueName>Values and has<valueName>Values.

You can specify a default value for a Stimulus value property. Instead of merely specifying the Stimulus type of the value, values = {visible: Boolean}, you can specify an object with the type and default, values = {visible: {type: Boolean, default: false}}. If you are declaring multiple value properties, you can declare some with defaults and some without.